Software Design Document

Digital Knowledge Platform (DKP)

Project F • Team: Semicolon Squad

Version 1.0 • 2026-04-15

Team	Semicolon Squad
Project	Digital Knowledge Platform (DKP)
Repository	github.com/NuruzzamanCS/digital-knowledge-platform
Commit / Tag	Sprint-1 / v1.0
Date	2026-04-15
Related SRS	v0.1 (2026-03-07)

Document prepared by: Faria Yasmin, Hasibul Islam Sifat, Md. Nuruzzaman, Yuki Bhuiyan

Based on SRS v0.1 dated 2026-03-07

Contents

1 System Overview **2**

2 Architecture Design **3**
2.1 High-Level Architecture 3
2.2 Component Breakdown 3
2.3 Key Design Decisions 4

3 Data Design **5**

4 Core Flow Design **7**

5 Security and Access Design **10**

6 Implementation Plan **11**

7 Testing Plan (Design Level) **13**

8 Exit Checklist **14**

1. System Overview

Problem Summary

Universities and research institutions in Bangladesh manage vast academic assets—archived documents, research papers, student projects, and physical library holdings—scattered across filing cabinets, disconnected drives, and legacy systems. This fragmentation causes discovery failure, knowledge loss (especially student projects), operational inefficiency in library management, access control gaps, and barriers to Bangla-language users.

Target Users

- **Public Visitors** — Browse public catalog, view published research and archives.
- **Students** — Search/download resources, borrow books, submit project showcases.
- **Researchers / Faculty** — Upload and manage research outputs, maintain lab portfolios.
- **Library Members** — Full borrowing workflow, wishlist, hold requests, history.
- **Staff / Archivists** — Upload/organise documents, bulk operations, metadata management.
- **Librarians** — Full catalog CRUD, lending/return, overdue/fine management, reports.
- **System Administrators** — User management, role configuration, audit logs.

Project Goals

- Provide a unified web platform with four modules: Digital Archive, Research Repository, Student Project Showcase, and Digital Library & Catalog.
- Support multi-format uploads (PDF, DOCX, audio, video, images) with rich bilingual metadata.
- Enable full-text and metadata search in both Bangla and English with proper tokenisation.
- Implement document versioning with lifecycle states: Draft → Review → Published → Archived.
- Enforce four access tiers: Public, Member, Staff, and Restricted.
- Deliver a complete lending/return workflow with due-date tracking, overdue notifications, and fine management.
- Support barcode/QR scanning for physical library operations.
- Provide bulk upload and bulk metadata editing tools for archivists and librarians.

Out of Scope

- E-commerce / online fine payment gateway (fines tracked only, not collected online in v1).
- AI-powered recommendation engine.
- Native mobile applications (iOS/Android) — responsive web only.
- Inter-library loan across institutions.
- Plagiarism detection.
- Real-time collaborative document editing.
- Integration with external LMS (Moodle, Canvas) in v1.

2. Architecture Design

2.1 High-Level Architecture

Main Components / Services

- **Frontend (React.js / Next.js)** — SPA with SSR for public pages; bilingual UI (EN/BN); responsive.
- **API Gateway / Backend (Node.js + Express or Django REST)** — Stateless REST API; JWT-based auth middleware; RBAC enforcement.
- **PostgreSQL** — Relational store for all structured data: users, items, transactions, fines.
- **Elasticsearch** — Full-text search engine with Bangla ICU analyser for archive, research, and catalog search.
- **S3-Compatible Object Storage (MinIO / AWS S3)** — File storage for all uploads up to 500 MB each.
- **Notification Service** — Email notifications for overdue reminders, hold availability, advisor approval.
- **Background Job Scheduler** — Daily fine calculation, overdue detection, reminder dispatch.

Data Flow Summary

Users interact with the React frontend, which calls the REST API over HTTPS. The API authenticates via JWT, checks RBAC, and routes requests to PostgreSQL (structured data) or Elasticsearch (search queries). File uploads are streamed directly to S3 storage, with metadata indexed in both PostgreSQL and Elasticsearch. Background jobs run nightly for fine calculations and overdue reminders.

External Integrations

- S3-compatible object storage (MinIO for self-hosted or AWS S3 for production).
- SMTP email provider for notifications.
- Barcode/QR library (e.g., ZXing or jsQR) for webcam and handheld scanner support.

2.2 Component Breakdown

Component	Responsibility	Inputs	Outputs	Notes
React Frontend	Render UI; handle forms, routing, state	User actions, API responses	REST API calls, DOM updates	Next.js for SSR on public pages; bilingual toggle
Auth Service	Issue & validate JWT tokens; RBAC middleware	Credentials, JWT	Access tokens, 401/403 responses	30-min inactivity timeout; bcrypt passwords

Component	Responsibility	Inputs	Outputs	Notes
Archive Module API	CRUD for archive items; versioning; access tier enforcement	File uploads, metadata, search queries	Archive items, search results, audit logs	Bulk upload up to 50 files; progress tracking
Research Repo API	Manage research outputs, lab portfolios, author profiles, citation export	Paper metadata, file, DOI	Research items, BibTeX/APA/MLA exports	Persistent DKP identifiers assigned on publish
Showcase API	Student project submission, advisor approval workflow, gallery	Project form, media files	Project records, notifications, gallery feed	Advisor notified via email on submission
Library Catalog API	Catalog CRUD, lending/return workflow, holds, fines, reports	Catalog data, barcode scans, member ID	Transactions, fines, reports (PDF/CSV)	Barcode/QR generation as printable PDF
Search Service	Full-text + faceted search across all modules	Search queries (EN/BN), filters	Ranked results with metadata	Elasticsearch with Bangla ICU analyser
File Storage	Store and serve uploaded files securely	Raw file bytes	Signed S3 URLs for access-controlled download	Chunked/resumable upload via tus protocol
Background Jobs	Daily fine calculation, overdue detection, reminder emails	Scheduled triggers, DB state	Fine records, notifications	Cron-based; Prometheus metrics
Audit Logger	Append-only log of all state-changing events	User ID, action, entity	Audit log entries in PostgreSQL	Immutable; admin-only read access

2.3 Key Design Decisions

Decision	Options Considered	Chosen Option	Reason
Backend Framework	Django REST, Node.js + Express, FastAPI	Node.js + Express (or Django)	Team familiarity; fast iteration; large ecosystem for file handling and barcode libs
Frontend Framework	React SPA, Next.js, Vue.js	Next.js (React-based)	SSR for public search pages improves SEO and load time; familiar to team
Search Engine	PostgreSQL full-text, Algolia, Elasticsearch	Elasticsearch with ICU plugin	Best Bangla tokenisation support; handles 1M+ documents at required speed
File Storage	Local filesystem, AWS S3, MinIO	S3-compatible (MinIO/AWS S3)	Scalable, access-controlled, supports presigned URLs; swappable between local and cloud
Authentication	Session-based, OAuth SSO, JWT	JWT with refresh tokens	Stateless API-friendly; 30-min timeout; SSO deferred to v2
File Upload Protocol	Standard multipart, tus resumable, chunked	Tus resumable upload protocol	Handles 500 MB files reliably; survives network interruptions; provides progress tracking
Access Control	ACL per item, flat role, RBAC	RBAC with 7 roles	Matches SRS requirement of 4 access tiers and 7 user roles; enforced at API middleware + DB level

3. Data Design

Core Entities

Entity	Key Fields	Purpose
User	user_id (UUID), email, password_hash, role, department, membership_status	All platform users across all roles (Guest to Admin)
ArchiveItem	item_id, title_en, title_bn, file_url, file_type, access_tier, status, version	Digital archive documents and media with versioning and access control

Entity	Key Fields	Purpose
ArchiveVersion	version_id, item_id (FK), version_number, file_url, metadata_snapshot, changed_by	Version history per archive item; stores each state snapshot
Tag	tag_id, name_en, name_bn	Bilingual categorisation tags applied M:N to archive items
ResearchOutput	output_id, title, abstract, authors (JSON), keywords, doi, file_url, lab_id (FK)	Research papers, reports, and datasets with DOI-style identifiers
Lab	lab_id, name, description, head_researcher (FK)	Research group portfolio pages
StudentProject	project_id, title, abstract, team_members (JSON), advisor_id (FK), semester, department, technologies	Student showcase submissions with approval workflow
CatalogItem	catalog_id, title, isbn, authors, publisher, total_copies, available_copies, barcode, shelf_location	Physical library catalog with copy tracking and barcode
LendingTransaction	transaction_id, catalog_id (FK), member_id (FK), issue_date, due_date, return_date, fine_amount	Book borrow/return records with fine tracking
HoldRequest	hold_id, catalog_id (FK), member_id (FK), request_date, status	FIFO hold queue per catalog item
Wishlist	wishlist_id, member_id (FK), catalog_id (FK), added_at	Member saved items (unique per member+item)
Fine	fine_id, member_id (FK), transaction_id (FK), amount, reason, status	Fine records linked to overdue lending transactions
AuditLog	log_id, user_id (FK), action, entity_type, entity_id, details (JSON), timestamp	Append-only, immutable event log for compliance and security
Notification	notification_id, user_id (FK), type, message, read, created_at	In-app and email notification records for reminders and approvals

Key Relationships

- **User → ArchiveItem (1:N)** — one user uploads many archive items.
- **ArchiveItem → ArchiveVersion (1:N)** — one item has many versions.
- **ArchiveItem ↔ Tag (M:N)** — via junction table `item_tags`.

- **User → ResearchOutput (1:N)** — one researcher publishes many outputs.
- **ResearchOutput → Lab (N:1)** — many outputs belong to one lab.
- **User → LendingTransaction → CatalogItem** — member borrows catalog items.
- **CatalogItem → HoldRequest (1:N)** — one item can have many holds (FIFO ordered).
- **User ↔ Wishlist ↔ CatalogItem (M:N)** — member saves many catalog items.
- **LendingTransaction → Fine (1:1)** — one overdue transaction generates one fine.

Constraints

- `email`: UNIQUE, NOT NULL, valid email format.
- `role`: ENUM (`guest`, `member`, `student_author`, `researcher`, `archivist`, `librarian`, `admin`).
- `access_tier`: ENUM (`public`, `member`, `staff`, `restricted`).
- `status` (ArchiveItem): ENUM (`draft`, `review`, `published`, `archived`).
- `isbn`: UNIQUE where not null; valid ISBN-10 or ISBN-13 format.
- `available_copies`: $0 \leq$ available_copies $\leq$ total_copies.
- `fine_amount`: DECIMAL(10,2), ≥ 0.00.
- `due_date`: must be $>$ `issue_date`.
- `file_size`: must be > 0 and $\leq 524{,}288{,}000$ bytes (500 MB).
- `Wishlist`: UNIQUE constraint on (`member_id`, `catalog_id`).
- `AuditLog`: append-only; no UPDATE or DELETE permitted.

Data Lifecycle Notes

- Archive items follow a strict lifecycle: Draft → Review → Published → Archived. Status regression is only allowed by Archivist or Admin.
- Deleted catalog items are soft-deleted (`deleted_at` timestamp) and recoverable within 30 days.
- Lending transactions are never deleted; `return_date` is populated on return and `fine_amount` calculated.
- User accounts support GDPR-style deletion: personal data anonymised, transactional records retained.
- Files in S3 are retained indefinitely; ArchiveVersion records link to each file version's S3 key.

4. Core Flow Design

Flow 1: Upload and Publish an Archive Document

Attribute	Details
Trigger	Authenticated Archivist or Librarian navigates to Archive → Upload and selects one or more files.
Step 1	Frontend validates file type (PDF, DOCX, PPTX, TXT, images, audio, video) and size ($\leq$ 500 MB) before upload begins.
Step 2	File is uploaded to S3 via tus resumable protocol; progress bar shown per file. On completion, S3 returns a file key.

Attribute	Details
Step 3	API creates an ArchiveItem record in PostgreSQL with status = 'draft' and stores the S3 key. Version 1.0 record created in ArchiveVersion.
Step 4	Metadata editor auto-populates with extracted data. Archivist fills in: title_en, title_bn, description, author(s), tags, category, language, access_tier.
Step 5	Archivist clicks 'Save as Draft.' System saves metadata; Elasticsearch indexes the item (draft items visible only to staff).
Step 6	Archivist or Reviewer changes status to 'Review.' Item enters review queue.
Step 7	Reviewer examines content and changes status to 'Published.' Item becomes discoverable to users matching its access tier.
Step 8	Audit log entry created for every state change (user_id, action, entity_id, timestamp).
Success Outcome	Document stored in S3, metadata in PostgreSQL, indexed in Elasticsearch. Version 1.0 recorded. Item visible in search per access tier.
Failure: Unsupported format	Frontend rejects file before upload; error message displayed.
Failure: File > 500 MB	Frontend rejects file; 'File too large' error displayed.
Failure: Network interruption	Tus protocol resumes upload from last byte; if session expires, user retries from interrupted point.
Failure: Reviewer requests changes	Status reverts to 'Draft'; author notified via in-app notification and email.

Flow 2: Borrow a Book (Lending Workflow)

Attribute	Details
Trigger	Library Member approaches the librarian desk with a physical book they want to borrow.
Step 1	Librarian opens the issue screen on the Librarian Dashboard.
Step 2	Librarian scans the book's barcode/QR code via webcam or handheld scanner. API identifies the CatalogItem.
Step 3	System checks `available_copies` > 0. If no copies available, librarian is prompted to place a hold for the member.

Attribute	Details
Step 4	Librarian scans or enters the member's ID. System retrieves the User record and verifies eligibility: active membership, no blocking fines, active borrows < 5.
Step 5	System displays member name, active borrows, and any outstanding fines for librarian confirmation.
Step 6	Librarian confirms the issue. System creates a LendingTransaction: borrower_id, item_id, issue_date = today, due_date = today + 14 days.
Step 7	`CatalogItem.available_copies` decremented by 1 atomically.
Step 8	Member's dashboard updated with new active borrow. Audit log entry created.
Step 9	System schedules email reminders: 3 days before due date and on due date.
Success Outcome	Transaction recorded in PostgreSQL. Member's active borrow list updated. Catalog availability decremented. Audit log entry created.
Failure: Barcode scan fails	Manual search fallback: librarian can search by title or ISBN to identify the item.
Failure: Member has overdue books	Warning displayed; librarian can proceed with override or block based on policy.
Failure: Borrow limit reached	'Borrow limit reached (max 5 items)' error; issue blocked.
Failure: Race condition (last copy taken)	Transaction fails; librarian offered 'Place Hold' option for the member.

Flow 3: Student Project Submission and Advisor Approval

Attribute	Details
Trigger	Authenticated Student Author navigates to Showcase → Submit Project during an active submission period.
Step 1	Student fills in: title, abstract, team members, advisor (selected from user list), semester, department, technologies (tags).
Step 2	Student uploads project report (PDF, required), optionally a demo video (MP4, ≤ 500 MB), and optionally a source code link.
Step 3	System validates all required fields and file formats. Validation errors shown inline.
Step 4	Student clicks 'Submit for Review.' System saves StudentProject with status = 'pending_review.'

Attribute	Details
Step 5	Advisor receives email notification with a link to the review queue.
Step 6	Advisor opens the submission, reviews title, abstract, report PDF, and video.
Step 7 (Approve)	Advisor clicks 'Approve.' Status changes to 'published.' Project appears in the public showcase gallery.
Step 7 (Changes)	Advisor clicks 'Request Changes' with comments. Status reverts to 'draft.' Student is notified with advisor's comments.
Success Outcome	Approved project publicly visible in showcase gallery. Student and advisor profiles updated.
Failure: Required field missing	Validation error highlighted; submission blocked until resolved.
Failure: File format or size error	File rejected; specific error message shown; other fields retained.

5. Security and Access Design

Authentication Approach

JWT (JSON Web Tokens) with short-lived access tokens (15 minutes) and longer-lived refresh tokens (7 days). Tokens are signed with HS256 and include `user_id` and `role` claims. A 30-minute inactivity timeout invalidates the session. Passwords are hashed using bcrypt (min 12 rounds) and must meet the policy: minimum 8 characters, at least 1 uppercase, 1 lowercase, 1 digit, 1 special character. SSO (Google Workspace/LDAP) is deferred to v2.

Authorization Model — RBAC

Role	Permissions
Guest	View public catalog; browse published archives/research/showcase; search (public items only); read-only.
Member	All Guest permissions + borrow/return books, hold requests, wishlist, borrowing history, download member-tier resources.
Student Author	All Member permissions + create/edit own project submissions, upload project media; editing locked after advisor approval.
Researcher	All Member permissions + upload/manage own research outputs, maintain lab portfolio, edit own research metadata.
Archivist / Staff	All Member permissions + upload/edit/delete archive items, bulk upload, bulk metadata editing, manage access tiers per item.

Role	Permissions
Librarian	All Archivist permissions + full catalog CRUD, manage lending/returns, track overdues, manage fines (waive/adjust), generate reports, import/export catalog.
Admin	All permissions + user management, role assignment, system configuration, view and export audit logs, backup management.

RBAC is enforced at two layers: (1) API middleware validates the JWT role claim before every request; (2) Database queries are scoped by user role to prevent privilege escalation. Access tier enforcement on archive items is checked at both the API and query level — a Member cannot retrieve a Staff-tier item even if they know the `item_id`.

Sensitive Data Handling

- All traffic encrypted via HTTPS (TLS 1.2+). HTTP redirected to HTTPS.
- Passwords stored exclusively as bcrypt hashes; never logged or transmitted in plaintext.
- Sensitive fields (email, fine amounts, borrowing history) encrypted at rest in the database.
- S3 files served via time-limited presigned URLs (15-minute expiry), not public URLs.
- File uploads scanned for malware; executable file types (EXE, BAT, SH, etc.) rejected at MIME type and magic-byte levels.
- Input validation and sanitisation applied to all API endpoints to prevent XSS, SQL injection, and path traversal attacks.
- Personal data deletable on member request (GDPR-style); transactional records anonymised rather than deleted.

Audit / Logging Plan

- All state-changing events written to the AuditLog table: user_id, action (CREATE, UPDATE, DELETE, ACCESS, LOGIN), entity_type, entity_id, details (JSON), timestamp.
- AuditLog is append-only: no UPDATE or DELETE SQL permitted; enforced via trigger.
- Admin role can view and filter audit logs by user, action type, entity, and date range.
- Application logs use structured JSON format (timestamp, level, request_id, user_id, action).
- Infrastructure metrics collected via Prometheus; dashboards in Grafana; alerts via email/Slack for error-rate spikes or downtime.
- Security events (failed logins, access denied on restricted items) logged at WARN level and monitored for anomalies.

6. Implementation Plan

Sprint 1 Completed Features

- **Project scaffolding**: monorepo setup, CI/CD pipeline (lint + test on push), Docker Compose for local dev.
- **Authentication**: user registration, login, JWT issuance/refresh, bcrypt password hashing, role-based middleware.
- **Digital Archive Module (core)**: single file upload to S3 (tus protocol), metadata editor (EN + BN fields), Draft/Published status, access tier enforcement, basic search via Elasticsearch.
- **Digital Library Catalog (core)**: catalog item CRUD for librarians, public search with availability display, basic lending/return transaction, due date calculation.
- **Database**: PostgreSQL schema migrations for all 14 entities, seed data for testing.
- **Elasticsearch**: index setup with Bangla ICU analyser; basic document indexing on archive item publish.

Pending Features for Sprint 2

- **Archive Module**: versioning (full lifecycle Draft → Review → Published → Archived), bulk upload (up to 50 files), bulk metadata editing, audit log integration, in-browser media viewer.
- **Research Repository**: research output upload, author profile pages, lab portfolio pages, DOI-style identifier assignment, citation export (BibTeX, APA, MLA).
- **Student Project Showcase**: project submission form with all fields, advisor approval workflow and email notifications, public gallery with pagination and filters.
- **Library Catalog**: barcode/QR generation and scanning, overdue fine auto-calculation (Tk 5/day), fine waiver by librarian, member borrowing history, hold request queue (FIFO), member wishlist, librarian dashboard with real-time stats, circulation reports (PDF/CSV), CSV/Excel catalog import with duplicate detection, book renewal workflow.
- **Notification Service**: email reminders 3 days before and on due date; hold availability notifications; advisor approval notifications.
- **Admin Panel**: user management, role assignment, audit log viewer.

Technical Risks

Risk	Probability	Impact	Mitigation
Bangla search tokenisation inaccurate with default Elasticsearch analysers	Medium	High	Configure Bangla-specific ICU analyser; test with sample corpus in Sprint 1 tech spike.
Large file uploads (500 MB) failing due to timeouts or memory	Medium	Medium	Tus resumable upload protocol; server streaming (no buffering in memory); retry on failure.

Risk	Probability	Impact	Mitigation
Barcode/QR webcam scanning unreliable in low-light conditions	Medium	Medium	Manual entry fallback; recommend USB handheld scanner for librarian desks.
Team inexperience with Elasticsearch, S3, barcode libraries	Medium	High	Tech spikes in Sprint 1 Week 1; pair programming; documented setup guides.
Large scope (4 modules) risks incomplete delivery in 14 weeks	High	High	Strict MoSCoW prioritisation; Archive + Library core in Sprints 1–2; Research + Showcase in Sprint 3.
Access tier bugs could expose restricted content	Low	Critical	Access checks enforced at API middleware AND DB query levels; dedicated security test suite.
Dirty/inconsistent data in CSV catalog migration	Medium	Medium	Per-row validation on import; duplicate detection; error summary shown before commit.

7. Testing Plan (Design Level)

Unit Testing Targets

- Auth Service: password hashing/verification, JWT issuance and validation, token expiry, role claim extraction.
- Access tier enforcement middleware: each of the 7 roles against each of the 4 access tiers (28 combinations).
- Fine calculation logic: correct daily rate (Tk 5/day), zero fine for on-time returns, waiver/adjustment application.
- File validation: accepted MIME types, size limits (under/over 500 MB), magic-byte checks for executable rejection.
- ISBN validation: ISBN-10 and ISBN-13 checksum verification.
- Archive status transitions: allowed and blocked transitions in the Draft → Review → Published → Archived lifecycle.
- Bangla text storage and retrieval: Unicode integrity through the full DB round-trip.

Integration Testing Targets

- File upload → S3 storage → metadata saved in PostgreSQL → Elasticsearch indexed (full upload flow).
- Lending issue flow: barcode scan → member eligibility check → transaction creation → available_copies decrement.
- Return flow: barcode scan → transaction update → available_copies increment → fine calculation if overdue.
- Advisor approval flow: student submits → advisor receives notification → approves → project appears in gallery.

- Hold queue: all copies on loan → hold placed → book returned → first-in-queue notified.
- CSV catalog import: valid rows imported, invalid rows reported per-row, duplicates detected.
- Overdue reminder scheduler: cron fires → detects overdue transactions → sends email notifications.

End-to-End (E2E) Testing Targets

- **TC-TXX-001**: Archivist uploads PDF → fills metadata → saves Draft → publishes → item appears in public search.
- **TC-TXX-007**: Member attempts to access a Restricted item → 'Access Denied' message shown, no file served.
- **TC-TXX-013**: Librarian scans barcode → enters member ID → confirms issue → member dashboard shows active borrow.
- **TC-TXX-014**: Librarian processes return 3 days overdue → system calculates fine of Tk 15 → fine appears on member account.
- **TC-TXX-009**: Student submits project with PDF and video → advisor approves → project appears in showcase gallery.
- **TC-TXX-012**: Public visitor searches catalog without login → sees availability → 'Borrow' button is hidden.
- **TC-TXX-017**: Member places hold on all-copies-on-loan book → book returned → member receives notification.

Known Hard-to-Test Areas

- **Bangla full-text search relevance ranking** — requires a curated test corpus of Bangla documents and manual relevance judgements.
- **Barcode/QR webcam scanning** — requires physical camera access; tested manually with real hardware; unit-tested with mocked scanner input.
- **Resumable upload recovery after network interruption** — requires controlled network simulation (e.g., toxiproxy); automated in CI with simulated packet loss.
- **Race condition on last available copy** — requires concurrent requests in integration tests; tested with Jest concurrent execution.
- **Email notification delivery** — mocked in unit tests using a fake SMTP server (Mailhog); verified end-to-end manually in staging environment.

8. Exit Checklist

✓ **System overview is complete** — Section 1 documents problem summary, target users, goals, and out-of-scope items.

✓ **Architecture and components are documented** — Section 2 provides high-level architecture, a 10-component breakdown table with responsibilities/inputs/outputs, and 7 key design decisions with rationale.

✓ **Data model and constraints are documented** — Section 3 covers 14 core entities, all key relationships, field constraints (types, uniqueness, nullability, enums), and data lifecycle rules.

✓ **At least 2 critical flows are documented** — Section 4 documents 3 flows:

Archive Upload & Publish, Book Lending, and Student Project Submission & Approval, each with trigger, steps, success outcome, and failure/edge cases.

✓ **Security and access model is defined** — Section 5 covers JWT authentication, 7-role RBAC model, sensitive data handling, encryption, file security, and audit/logging plan.

✓ **Sprint implementation and risk plan is included** — Section 6 lists Sprint 1 completed features, Sprint 2 pending features, 7 identified technical risks with probability/impact/mitigation.

Document prepared by: **Semicolon Squad** (Faria Yasmin, Hasibul Islam Sifat, Md. Nuruzzaman, Yuki Bhuiyan)
Based on SRS v0.1 dated 2026-03-07